

Lab2-Report

Step 0: Environment Setup

0a: Start the Kafka Cluster

To start the Kafka cluster environment that was configured in the previous session.

```
cd kafka-lab
docker compose up -d
docker compose ps
```

```
(bigdata) PS D:\EFREI\Data_Engineering> cd kafka-lab
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> docker compose up -d
time="2026-05-12T10:22:45+08:00" level=warning msg="D:\EFREI\Data_Engineering\kafka-lab\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it
to avoid potential confusion"
[+] Running 1/2
[+] Running 4/7ka-lab default Created 0.1s
[+] Network kafka-lab default Created 0.1s
[+] Volume kafka-lab_kafka1_data Creating 0.0s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Running 4/7
[+] Network kafka-lab default Created 0.1s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Running 7/8
[+] Network kafka-lab default Created 0.1s
[+] Container kafka1 Creating 0.1s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Volume kafka-lab_kafka2_data Created 0.0s
[+] Container kafka1 Creating 0.2s
[+] Running 7/8a-lab kafka3_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Container kafka2 Created 0.2s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka1 Created 0.3s
[+] Volume kafka-lab_kafka2_data Created 0.0s
[+] Container kafka3 Created 0.3s
[+] Running 5/8a-lab kafka3_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Container kafka2 Created 0.2s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka1 Created 0.3s
[+] Running 5/8a-lab kafka2_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Volume kafka-lab_kafka3_data Created 0.0s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka2 Starting 0.5s
[+] Running 5/8a-lab kafka2_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Volume kafka-lab_kafka3_data Created 0.0s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka2 Starting 0.6s
[+] Running 5/8a-lab kafka2_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Volume kafka-lab_kafka3_data Created 0.0s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka2 Starting 0.7s
[+] Running 5/8a-lab kafka2_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Volume kafka-lab_kafka3_data Created 0.0s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka2 Starting 0.8s
[+] Running 7/8a-lab kafka2_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka1 Starting 1.0s
[+] Volume kafka-lab_kafka2_data Created 0.0s
[+] Running 7/8a-lab kafka3_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Container kafka2 Starting 1.0s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka1 Starting 1.0s
[+] Volume kafka-lab_kafka2_data Created 0.0s
[+] Container kafka3 Starting 1.0s
[+] Running 8/8a-lab kafka3_data Created 0.0s
[+] Network kafka-lab default Created 0.1s
[+] Container kafka2 Starting 1.0s
[+] Volume kafka-lab_kafka1_data Created 0.0s
[+] Container kafka1 Starting 1.0s
[+] Volume kafka-lab_kafka2_data Created 0.0s
[+] Container kafka3 Starting 1.0s
[+] Volume kafka-lab_kafka3_data Created 0.0s
[+] Container kafka-ui Starting 0.9s
[+] Container kafka2 Starting 1.0s
[+] Container kafka1 Starting 1.0s
[+] Container kafka3 Starting 1.0s
[+] Container kafka-ui Starting 1.0s
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> docker compose ps
time="2026-05-12T10:22:53+08:00" level=warning msg="D:\EFREI\Data_Engineering\kafka-lab\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it
to avoid potential confusion"
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
kafka-ui provectuslabs/kafka-ui:latest /bin/sh -c 'java -... kafka-ui 8 seconds ago Up 6 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
kafka1 confluentinc/cp-kafka:7.5.0 /etc/confluent/dock... kafka1 8 seconds ago Up 7 seconds 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
kafka2 confluentinc/cp-kafka:7.5.0 /etc/confluent/dock... kafka2 8 seconds ago Up 7 seconds 0.0.0.0:9094->9094/tcp, [::]:9094->9094/tcp
kafka3 confluentinc/cp-kafka:7.5.0 /etc/confluent/dock... kafka3 8 seconds ago Up 7 seconds 0.0.0.0:9096->9096/tcp, [::]:9096->9096/tcp
```

0b: Create a Virtual Environment and Install the Library

Since I initially operated in a virtual environment, I can skip creating the virtual environment and directly install kafka-python-ng using pip.

```
pip install kafka-python-ng
```

```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> pip install kafka-python-ng
Collecting kafka-python-ng
  Downloading kafka_python_ng-2.2.3-py2.py3-none-any.whl.metadata (9.7 kB)
  Downloading kafka_python_ng-2.2.3-py2.py3-none-any.whl (232 kB)
Installing collected packages: kafka-python-ng
Successfully installed kafka-python-ng-2.2.3
```

0c: Create the Topic for This Lab

To create a new topic specifically designed for simulating IoT sensor events in this lab.

```
docker exec -it kafka1 bash
kafka-topics --bootstrap-server kafka1:29092 \
  --create \
  --topic sensor-events \
  --partitions 3 \
  --replication-factor 3

kafka-topics --bootstrap-server kafka1:29092 \
  --describe --topic sensor-events
```

```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> docker exec -it kafka1 bash
lappuser@kafka1 ~]$ kafka-topics --bootstrap-server kafka1:29092 \
> --create \
> --topic sensor-events \
> --partitions 3 \
> --replication-factor 3
Created topic sensor-events.
lappuser@kafka1 ~]$ kafka-topics --bootstrap-server kafka1:29092 \
> --describe --topic sensor-events
Topic: sensor-events   TopicId: R4ApP-_bQ5Su6scZ-j2e5g PartitionCount: 3      ReplicationFactor: 3      Configs: min.insync.replicas=2
Topic: sensor-events   Partition: 0    Leader: 2       Replicas: 2,3,1 Isr: 2,3,1
Topic: sensor-events   Partition: 1    Leader: 3       Replicas: 3,1,2 Isr: 3,1,2
Topic: sensor-events   Partition: 2    Leader: 1       Replicas: 1,2,3 Isr: 1,2,3
```

The script accesses the `kafka1` container and uses the CLI to create a topic named `sensor-events` with 3 partitions and a replication factor of 3. It then describes the topic to confirm its creation.

Step 1: Write the Producer

1a: Imports and Configuration

Create a Python file named `producer` using VS Code.

To import the required libraries and define the fundamental tools for the producer script.

```
from kafka import KafkaProducer
import json
import time
import random
```

```
producer.py > send_reading
1  # 1a Imports and Configuration
2  from kafka import KafkaProducer
3  import json
4  import time
5  import random
6
```

1b: Instantiate the Producer with Reliability Settings

To configure the `KafkaProducer` instance with specific settings targeting high durability and optimal throughput.

```
producer = KafkaProducer(
    bootstrap_servers=['localhost:9092', 'localhost:9094', 'localhost:9096'],
    acks='all',
    retries=5,
    max_in_flight_requests_per_connection=1,
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    key_serializer=lambda k: k.encode('utf-8'),
    linger_ms=10,
    batch_size=32768,
)
```

```
7  # 1b Instantiate the Producer with Reliability Settings
8  # Configure the Kafka producer with appropriate settings for reliability and performance
9  producer = KafkaProducer(
10     bootstrap_servers=['localhost:9092', 'localhost:9094', 'localhost:9096'],
11     acks='all',
12     retries=5,
13     max_in_flight_requests_per_connection=1,
14     value_serializer=lambda v: json.dumps(v).encode('utf-8'),
15     key_serializer=lambda k: k.encode('utf-8'),
16     linger_ms=10,
17     batch_size=32768,
18 )
19
```

Because the `kafka-python-ng` library does not support the `enable_idempotence` parameter natively, we must rely on `acks='all'` and `retries=5` to achieve robust at-least-once delivery guarantees.

1c: The Sending Loop

To continuously generate and send mock sensor data to the Kafka topic while handling acknowledgments gracefully.

```
SENSOR_TYPES = ['temperature', 'humidity', 'pressure']
TOPIC = 'sensor-events'

def send_reading(sensor_type, value):
    record = {
        'sensor': sensor_type,
        'value': value,
        'unit': {'temperature': 'C', 'humidity': '%', 'pressure': 'hPa'}[sensor_type],
        'timestamp': int(time.time() * 1000),
    }
    future = producer.send(
        topic=TOPIC,
        key=sensor_type,
        value=record,
    )
    return future

try:
    for i in range(50):
        sensor = random.choice(SENSOR_TYPES)
        value = round(random.uniform(10, 40), 2)
        future = send_reading(sensor, value)

        # Block until broker confirms receipt
        metadata = future.get(timeout=10)
        print(f"Sent to partition {metadata.partition}, offset {metadata.offset}")
        time.sleep(0.2)
finally:
    producer.flush() # send any remaining buffered records
    producer.close()
    print("Producer closed.")
```

```

20 # 1c The Sending Loop
21 # Simulate sensor readings for temperature, humidity, and pressure
22 SENSOR_TYPES = ['temperature', 'humidity', 'pressure']
23 TOPIC = 'sensor-events'
24
25 def send_reading(sensor_type, value):
26     record = {
27         'sensor': sensor_type,
28         'value': value,
29         'unit': {'temperature': 'C', 'humidity': '%', 'pressure': 'hPa'}[sensor_type],
30         'timestamp': int(time.time() * 1000),
31     }
32     future = producer.send(
33         topic=TOPIC,
34         key=sensor_type,
35         value=record,
36     )
37     return future
38
39 try:
40     for i in range(50):
41         sensor = random.choice(SENSOR_TYPES)
42         value = round(random.uniform(10, 40), 2)
43         future = send_reading(sensor, value)
44
45         # Block until broker confirms receipt
46         metadata = future.get(timeout=10)
47         print(f"Sent to partition {metadata.partition}, offset {metadata.offset}")
48         time.sleep(0.2)
49 finally:
50     producer.flush() # send any remaining buffered records
51     producer.close()
52     print("Producer closed.")

```

The code defines an array of sensor types and loops 50 times to generate random readings. `producer.send()` is asynchronous, returning a Future object immediately. The `.get(timeout=10)` method blocks the execution until the broker acknowledges the write. The `finally` block ensures that `producer.flush()` drains the internal buffer so no records are lost before the script exits.

Passing `key=sensor_type` ensures that all readings from a specific sensor type will consistently land on the exact same partition. This mechanism guarantees that a downstream consumer reading temperature data will observe the events in chronological order.

Step 2: Write the Consumer

2a: Imports and Configuration

Create a Python file named `consumer` using VS Code.

To import tools necessary for subscribing to topics, fetching records, and manually assigning partitions.

```

from kafka import KafkaConsumer, TopicPartition
import json

```

```

consumer.py > ...
1  # 2a: Imports and Configuration
2  from kafka import KafkaConsumer, TopicPartition
3  import json
4

```

2b: Instantiate the Consumer

To initialize the consumer with settings designed for manual offset control and consumer group scaling.

```

consumer = KafkaConsumer(
    'sensor-events',
    bootstrap_servers=['localhost:9092', 'localhost:9094', 'localhost:9096'],
    group_id='sensor-analytics',
    auto_offset_reset='earliest',
    enable_auto_commit=False,
    value_deserializer=lambda b: json.loads(b.decode('utf-8')),
    key_deserializer=lambda b: b.decode('utf-8') if b else None,
    max_poll_records=10,
    session_timeout_ms=45000,
    heartbeat_interval_ms=15000,
)

```

```

4
5  # 2b: Instantiate the Consumer
6  # Configure the Kafka consumer with appropriate settings for reliability and performance
7  consumer = KafkaConsumer(
8      'sensor-events',
9      bootstrap_servers=['localhost:9092', 'localhost:9094', 'localhost:9096'],
10     group_id='sensor-analytics',
11     auto_offset_reset='earliest',
12     enable_auto_commit=False,
13     value_deserializer=lambda b: json.loads(b.decode('utf-8')),
14     key_deserializer=lambda b: b.decode('utf-8') if b else None,
15     max_poll_records=10,
16     session_timeout_ms=45000,
17     heartbeat_interval_ms=15000,
18 )
19

```

Setting `enable_auto_commit=False` is critical for achieving "at-least-once" delivery semantics by shifting the commit responsibility to the application layer.

2c: The Processing Loop with Manual Commit

To continuously poll for new messages, process them (e.g., alert on high temperatures), and manually commit offsets.

```

print("Consumer started. Waiting for messages...")
try:
    for message in consumer:
        record = message.value
        key = message.key
        part = message.partition

```

```

        offset = message.offset

        # Business logic
        print(f"[P{part}|O{offset}] sensor={key} value={record['value']}{record['unit']}")

        if record['sensor'] == 'temperature' and record['value'] > 35:
            print(f" >>> ALERT: high temperature {record['value']} C!")

        # Commit AFTER processing
        consumer.commit()
    except KeyboardInterrupt:
        print("Stopping consumer...")
    finally:
        consumer.close()
        print("Consumer closed.")

```

```

20 # 2c: The Processing Loop with Manual Commit
21 # Start consuming messages from the 'sensor-events' topic and process them according to the business logic
22 print("Consumer started. Waiting for messages...")
23 try:
24     for message in consumer:
25         record = message.value
26         key = message.key
27         part = message.partition
28         offset = message.offset
29
30         # Business logic
31         print(f"[P{part}|O{offset}] sensor={key} value={record['value']}{record['unit']}")
32
33         if record['sensor'] == 'temperature' and record['value'] > 35:
34             print(f" >>> ALERT: high temperature {record['value']} C!")
35
36         # Commit AFTER processing
37         consumer.commit()
38 except KeyboardInterrupt:
39     print("Stopping consumer...")
40 finally:
41     consumer.close()
42     print("Consumer closed.")
43

```

The `for message in consumer:` line internally calls `poll()` and yields individual records. The

`consumer.commit()` function is invoked only *after* the record is fully processed and printed.

Because we commit the offset after processing, this implements an at-least-once pattern. If the application crashes between processing the message and issuing the commit command, the message will be replayed upon restart.

Step 3: Run the Pipeline End-to-End

3a: Start the Consumer First

To have the consumer actively listening before messages are produced, allowing real-time observation.

```

# Terminal 1
python consumer.py

```

```
kafka-1.0.0/conf/clients/ep_kafka1.0.0/.../etc/conf/clients/...  
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py  
Consumer started. Waiting for messages...
```

Starting the consumer first ensures no messages are missed in real-time observation.

3b: Start the Producer

To inject 50 generated messages into the Kafka topic.

```
# Terminal 2  
python producer.py
```

The producer runs its 50-iteration loop, sending data to the cluster.

Terminal 2


```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python producer.py
```

```
Sent to partition 2, offset 0
Sent to partition 0, offset 0
Sent to partition 2, offset 1
Sent to partition 0, offset 1
Sent to partition 2, offset 2
Sent to partition 2, offset 3
Sent to partition 2, offset 4
Sent to partition 0, offset 2
Sent to partition 0, offset 3
Sent to partition 2, offset 5
Sent to partition 2, offset 6
Sent to partition 2, offset 7
Sent to partition 2, offset 8
Sent to partition 0, offset 4
Sent to partition 0, offset 5
Sent to partition 2, offset 9
Sent to partition 2, offset 10
Sent to partition 2, offset 11
Sent to partition 2, offset 12
Sent to partition 2, offset 13
Sent to partition 2, offset 14
Sent to partition 2, offset 15
Sent to partition 0, offset 6
Sent to partition 2, offset 16
Sent to partition 2, offset 17
Sent to partition 2, offset 18
Sent to partition 0, offset 7
Sent to partition 2, offset 19
Sent to partition 2, offset 20
Sent to partition 2, offset 21
Sent to partition 0, offset 8
Sent to partition 0, offset 9
Sent to partition 0, offset 10
Sent to partition 2, offset 22
Sent to partition 2, offset 23
Sent to partition 2, offset 24
Sent to partition 0, offset 11
Sent to partition 2, offset 25
Sent to partition 0, offset 12
Sent to partition 0, offset 13
Sent to partition 2, offset 26
Sent to partition 2, offset 27
Sent to partition 2, offset 28
Sent to partition 0, offset 14
Sent to partition 0, offset 15
Sent to partition 2, offset 29
Sent to partition 2, offset 30
Sent to partition 2, offset 31
Sent to partition 2, offset 32
```

```
sent to partition 0, offset 10
Producer closed.
(1; 1; 1) \ PC.D \ PERPI \ P ... F ... i ... i \ 1; 1; 1; 1)
```

```
Producer closed.
```

[illegible]

Terminal 1

```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py
Consumer started. Waiting for messages...
[P2 00] sensor=pressure value=18.71hPa
[P0 00] sensor=humidity value=30.88%
[P2 01] sensor=pressure value=28.58hPa
[P0 01] sensor=humidity value=33.27%
[P2 02] sensor=temperature value=10.9C
[P2 03] sensor=pressure value=35.04hPa
[P2 04] sensor=temperature value=22.78C
[P0 02] sensor=humidity value=31.93%
[P0 03] sensor=humidity value=13.14%
[P2 05] sensor=temperature value=26.05C
[P2 06] sensor=temperature value=37.77C
>>> ALERT: high temperature 37.77 C!
[P2 07] sensor=pressure value=35.47hPa
[P2 08] sensor=temperature value=15.81C
[P0 04] sensor=humidity value=14.64%
[P0 05] sensor=humidity value=10.74%
[P2 09] sensor=temperature value=36.09C
>>> ALERT: high temperature 36.09 C!
[P2 010] sensor=pressure value=26.6hPa
[P2 011] sensor=pressure value=26.16hPa
[P2 012] sensor=temperature value=28.76C
[P2 013] sensor=pressure value=29.12hPa
[P2 014] sensor=pressure value=13.69hPa
[P2 015] sensor=temperature value=17.96C
[P0 06] sensor=humidity value=11.26%
[P2 016] sensor=temperature value=34.59C
[P2 017] sensor=pressure value=26.56hPa
[P2 018] sensor=pressure value=34.31hPa
[P0 07] sensor=humidity value=16.1%
[P2 019] sensor=temperature value=14.02C
[P2 020] sensor=temperature value=26.04C
[P2 021] sensor=temperature value=21.22C
[P0 08] sensor=humidity value=12.02%
[P0 09] sensor=humidity value=28.43%
[P0 010] sensor=humidity value=27.62%
[P2 022] sensor=pressure value=32.55hPa
[P2 023] sensor=temperature value=32.09C
[P2 024] sensor=temperature value=15.73C
[P0 011] sensor=humidity value=32.93%
[P2 025] sensor=temperature value=32.02C
[P0 012] sensor=humidity value=31.54%
[P0 013] sensor=humidity value=31.74%
[P2 026] sensor=pressure value=33.74hPa
[P2 027] sensor=temperature value=30.04C
[P2 028] sensor=temperature value=35.18C
>>> ALERT: high temperature 35.18 C!
[P0 014] sensor=humidity value=17.25%
[P0 015] sensor=humidity value=12.16%
```

```
[P0 015] sensor=humidity value=12.10%  
[P2 029] sensor=temperature value=19.98C  
[P2 030] sensor=temperature value=21.66C  
[P2 031] sensor=pressure value=20.0hPa  
[P2 032] sensor=pressure value=39.51hPa  
[P0 016] sensor=humidity value=33.06%
```

Terminal 2 shows "Sent to partition X, offset Y". Simultaneously, Terminal 1 prints the processed records and high-temperature alerts. By observing the output in Terminal 1, it can verify that all temperature readings land on the identical partition, all humidity readings land on another identical partition, and offsets strictly increase. Temperature values above 35°C trigger the alert message.

Step 4: Consumer Group Scaling Exercise

4a: Start a Second Consumer Instance

To observe how Kafka scales consumption by distributing partitions among multiple consumers in the same group.

```
# Terminal 3 (same consumer.py, same group_id)  
python consumer.py
```

Run producer.py again

```
# Terminal 2  
python producer.py
```

Terminal 1

```
[P0|017] sensor=humidity value=10.77%
[P0|018] sensor=humidity value=25.16%
[P0|019] sensor=humidity value=16.1%
[P0|020] sensor=humidity value=32.45%
[P0|021] sensor=humidity value=35.03%
[P0|022] sensor=humidity value=27.41%
[P0|023] sensor=humidity value=29.15%
[P0|024] sensor=humidity value=23.34%
[P0|025] sensor=humidity value=18.14%
[P0|026] sensor=humidity value=20.32%
[P0|027] sensor=humidity value=18.94%
[P0|028] sensor=humidity value=31.38%
[P0|029] sensor=humidity value=11.24%
[P0|030] sensor=humidity value=13.38%
[P0|031] sensor=humidity value=37.52%
[P0|032] sensor=humidity value=21.54%
[P0|033] sensor=humidity value=27.46%
[P0|034] sensor=humidity value=14.26%
[P0|035] sensor=humidity value=19.16%
```

Terminal 3

```

kafka-1.0.0-rc1\bin\windows\bin /etc/ssl/certs/ssl-certificates
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py
Consumer started. Waiting for messages...
[P2|033] sensor=pressure value=11.83hPa
[P2|034] sensor=temperature value=24.07C
[P2|035] sensor=pressure value=22.94hPa
[P2|036] sensor=pressure value=39.63hPa
[P2|037] sensor=temperature value=29.93C
[P2|038] sensor=temperature value=23.93C
[P2|039] sensor=pressure value=19.43hPa
[P2|040] sensor=pressure value=10.05hPa
[P2|041] sensor=pressure value=17.23hPa
[P2|042] sensor=pressure value=39.38hPa
[P2|043] sensor=pressure value=28.69hPa
[P2|044] sensor=temperature value=29.91C
[P2|045] sensor=temperature value=28.5C
[P2|046] sensor=temperature value=32.73C
[P2|047] sensor=pressure value=13.65hPa
[P2|048] sensor=temperature value=14.78C
[P2|049] sensor=pressure value=21.83hPa
[P2|050] sensor=temperature value=38.71C
    >>> ALERT: high temperature 38.71 C!
[P2|051] sensor=temperature value=34.63C
[P2|052] sensor=temperature value=11.2C
[P2|053] sensor=pressure value=26.31hPa
[P2|054] sensor=pressure value=27.93hPa
[P2|055] sensor=pressure value=12.44hPa
[P2|056] sensor=pressure value=35.15hPa
[P2|057] sensor=temperature value=36.0C
    >>> ALERT: high temperature 36.0 C!
[P2|058] sensor=temperature value=30.45C
[P2|059] sensor=temperature value=11.16C
[P2|060] sensor=temperature value=16.85C
[P2|061] sensor=temperature value=36.08C
    >>> ALERT: high temperature 36.08 C!
[P2|062] sensor=pressure value=13.1hPa
[P2|063] sensor=temperature value=38.11C
    >>> ALERT: high temperature 38.11 C!

```

Terminal 1 and Terminal 3 begin to share the incoming traffic. Each terminal only prints messages from its specially assigned partition. Kafka triggers a rebalance and redistributes the 3 partitions across the 2 active consumers, ensuring each partition is exclusively read by one consumer.

Why isn't P1 included?

The hash calculation and modulo 3 for the words "pressure" and "temperature" both result in 2. They are both crammed into partition P2.

4b: Check Partition Assignment

To programmatically verify which partitions are assigned to which consumer.

(Add to `consumer.py` right before the main processing loop)

```
# Force a poll to trigger partition assignment
consumer.poll(timeout_ms=1000)
assigned = consumer.assignment()
print(f"Assigned partitions: {[f'P{p.partition}' for p in assigned]}")
```

```
20 # check Partition Assignment
21 # Force a poll to trigger partition assignment
22 consumer.poll(timeout_ms=1000)
23 assigned = consumer.assignment()
24 print(f"Assigned partitions: {[f'P{p.partition}' for p in assigned]}")
25
```

`consumer.poll()` is forced to trigger the initial communication with the broker. `consumer.assignment()` fetches the set of assigned `TopicPartition` objects.

Terminal 1


```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py
Assigned partitions: ['P1', 'P2', 'P0']
Consumer started. Waiting for messages...
[P0 053] sensor=humidity value=31.2%
[P2 097] sensor=pressure value=25.83hPa
[P2 098] sensor=pressure value=28.15hPa
[P2 099] sensor=pressure value=13.5hPa
[P2 0100] sensor=pressure value=39.82hPa
[P0 054] sensor=humidity value=26.04%
[P0 055] sensor=humidity value=21.32%
[P2 0101] sensor=pressure value=11.75hPa
[P0 056] sensor=humidity value=24.34%
[P0 057] sensor=humidity value=31.38%
[P2 0102] sensor=pressure value=13.09hPa
[P2 0103] sensor=pressure value=10.22hPa
[P2 0104] sensor=temperature value=17.01C
[P0 058] sensor=humidity value=37.13%
[P0 059] sensor=humidity value=39.92%
[P2 0105] sensor=pressure value=22.95hPa
[P0 060] sensor=humidity value=31.64%
[P0 061] sensor=humidity value=17.19%
[P0 062] sensor=humidity value=31.28%
[P0 063] sensor=humidity value=12.04%
[P2 0106] sensor=temperature value=33.42C
[P2 0107] sensor=pressure value=13.2hPa
[P2 0108] sensor=temperature value=38.68C
>>> ALERT: high temperature 38.68 C!
[P2 0109] sensor=temperature value=23.93C
[P2 0110] sensor=temperature value=24.97C
[P2 0111] sensor=pressure value=35.04hPa
[P0 064] sensor=humidity value=32.55%
[P2 0112] sensor=pressure value=17.44hPa
[P0 065] sensor=humidity value=39.51%
[P0 066] sensor=humidity value=14.33%
[P0 067] sensor=humidity value=32.57%
[P0 068] sensor=humidity value=38.56%
[P0 069] sensor=humidity value=38.91%
[P0 070] sensor=humidity value=17.56%
```



```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py
Assigned partitions: ['P2']
Consumer started. Waiting for messages...
[P2|0113] sensor=temperature value=16.84C
[P2|0114] sensor=temperature value=12.88C
[P2|0115] sensor=temperature value=27.69C
[P2|0116] sensor=temperature value=32.87C
[P2|0117] sensor=pressure value=11.51hPa
[P2|0118] sensor=temperature value=37.1C
>>> ALERT: high temperature 37.1 C!
[P2|0119] sensor=temperature value=33.58C
[P2|0120] sensor=pressure value=19.63hPa
[P2|0121] sensor=temperature value=11.94C
[P2|0122] sensor=pressure value=10.52hPa
[P2|0123] sensor=pressure value=24.41hPa
[P2|0124] sensor=pressure value=15.3hPa
[P2|0125] sensor=pressure value=24.32hPa
[P2|0126] sensor=temperature value=36.24C
>>> ALERT: high temperature 36.24 C!
[P2|0127] sensor=temperature value=31.53C
[P2|0128] sensor=pressure value=11.24hPa
```

First, run Terminal 1. At the beginning of the terminal, you can see that it includes all the partitions. While running Terminal 1, open Terminal 3. You will find that only the p0 partition is generated after Terminal 1, while Terminal 3 generates another partition, p2.

4c: Observe the Rebalance on Consumer Exit

To see how Kafka handles a consumer leaving the group.

Press **Ctrl+C** in Terminal 3.

Terminal 3

```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py
Assigned partitions: ['P0', 'P1']
Consumer started. Waiting for messages...
[P0|071] sensor=humidity value=37.35%
[P0|072] sensor=humidity value=10.2%
[P0|073] sensor=humidity value=22.38%
[P0|074] sensor=humidity value=16.76%
[P0|075] sensor=humidity value=37.7%
[P0|076] sensor=humidity value=34.07%
[P0|077] sensor=humidity value=38.96%
Stopping consumer...
Consumer closed.
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab>
```

Terminal 1

```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py
Assigned partitions: ['P2']
Consumer started. Waiting for messages...
[P2|0129] sensor=temperature value=34.15C
[P2|0130] sensor=temperature value=16.42C
[P2|0131] sensor=temperature value=20.58C
[P2|0132] sensor=temperature value=21.32C
[P2|0133] sensor=temperature value=19.93C
[P2|0134] sensor=pressure value=12.31hPa
[P2|0135] sensor=pressure value=15.47hPa
[P2|0136] sensor=pressure value=32.53hPa
[P2|0137] sensor=temperature value=30.97C
[P2|0138] sensor=temperature value=17.52C
[P2|0139] sensor=temperature value=20.21C
[P2|0140] sensor=pressure value=25.21hPa
[P2|0141] sensor=pressure value=16.97hPa
[P2|0142] sensor=pressure value=31.52hPa
[P2|0143] sensor=temperature value=25.36C
[P2|0144] sensor=temperature value=39.16C
>>> ALERT: high temperature 39.16 C!
[P2|0145] sensor=temperature value=29.36C
[P2|0146] sensor=pressure value=24.04hPa
[P2|0147] sensor=pressure value=33.74hPa
[P2|0148] sensor=pressure value=21.64hPa
[P2|0149] sensor=temperature value=23.12C
[P2|0150] sensor=pressure value=35.56hPa
[P2|0151] sensor=pressure value=19.62hPa
[P2|0152] sensor=temperature value=18.53C
[P2|0153] sensor=temperature value=17.16C
[P2|0154] sensor=temperature value=15.75C
[P2|0155] sensor=temperature value=36.97C
>>> ALERT: high temperature 36.97 C!
[P0|078] sensor=humidity value=25.99%
[P0|079] sensor=humidity value=22.07%
[P0|080] sensor=humidity value=38.66%
[P0|081] sensor=humidity value=22.66%
[P0|082] sensor=humidity value=28.47%
[P0|083] sensor=humidity value=24.69%
[P0|084] sensor=humidity value=14.02%
[P0|085] sensor=humidity value=37.04%
[P0|086] sensor=humidity value=28.3%
[P0|087] sensor=humidity value=11.73%
[P0|088] sensor=humidity value=26.89%
[P2|0156] sensor=temperature value=33.18C
[P2|0157] sensor=pressure value=13.78hPa
[P0|089] sensor=humidity value=27.8%
[P2|0158] sensor=pressure value=37.25hPa
[P0|090] sensor=humidity value=38.84%
```

Initially, both terminals were working, each responsible for its own partitions. When I disconnected terminal 3, Kafka declared terminal 3 dead and triggered a new rebalancing, forcibly assigning tasks originally belonging to terminal 3 to the only surviving terminal 1. Terminal 1 then began generating all partitions.

Step 5: Fault Injection: Crash and Replay

5a: Produce More Messages

To populate the topic with a fresh batch of data to test fault tolerance.

```
python producer.py
```

5b: Crash the Consumer Mid-Processing

To simulate a sudden application failure before offsets can be manually committed.

```
python consumer.py  
# Let it print 3-4 messages, then press Ctrl+C quickly
```

5c: Restart and Observe Replay

To verify that uncommitted messages are safely re-delivered.

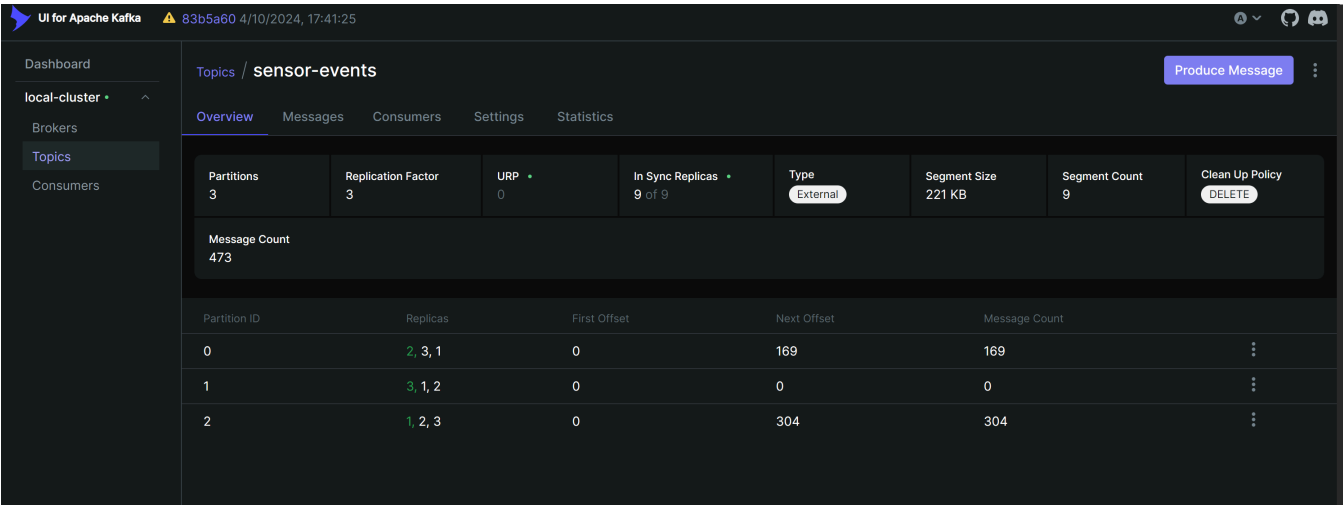
```
python consumer.py
```

```
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py
Assigned partitions: ['P0', 'P1', 'P2']
Consumer started. Waiting for messages...
[P0|091] sensor=humidity value=10.51%
[P2|0159] sensor=temperature value=23.84C
[P2|0160] sensor=temperature value=11.88C
[P0|092] sensor=humidity value=11.81%
[P2|0161] sensor=temperature value=12.63C
[P2|0162] sensor=temperature value=10.61C
[P2|0163] sensor=temperature value=12.7C
[P2|0164] sensor=temperature value=24.35C
[P0|093] sensor=humidity value=25.55%
[P2|0165] sensor=pressure value=19.97hPa
Stopping consumer...
Consumer closed.
(bigdata) PS D:\EFREI\Data_Engineering\kafka-lab> python consumer.py
Assigned partitions: ['P0', 'P1', 'P2']
Consumer started. Waiting for messages...
[P2|0166] sensor=temperature value=39.83C
>>> ALERT: high temperature 39.83 C!
[P2|0167] sensor=pressure value=25.23hPa
[P2|0168] sensor=pressure value=25.37hPa
[P2|0169] sensor=pressure value=25.67hPa
[P2|0170] sensor=temperature value=12.12C
[P2|0171] sensor=temperature value=28.34C
[P2|0172] sensor=temperature value=23.57C
[P2|0173] sensor=pressure value=14.34hPa
[P2|0174] sensor=temperature value=23.21C
[P2|0175] sensor=pressure value=24.22hPa
[P2|0176] sensor=temperature value=26.38C
[P2|0177] sensor=temperature value=11.07C
[P2|0178] sensor=temperature value=34.56C
[P2|0179] sensor=temperature value=12.87C
[P2|0180] sensor=pressure value=36.06hPa
[P2|0181] sensor=temperature value=13.85C
[P2|0182] sensor=temperature value=20.13C
[P2|0183] sensor=pressure value=25.14hPa
[P2|0184] sensor=temperature value=24.28C
[P0|0100] sensor=humidity value=31.89%
[P2|0185] sensor=pressure value=23.5hPa
[P0|0101] sensor=humidity value=24.72%
[P2|0186] sensor=temperature value=37.39C
>>> ALERT: high temperature 37.39 C!
[P2|0187] sensor=temperature value=20.08C
[P0|0102] sensor=humidity value=18.09%
[P0|0103] sensor=humidity value=34.12%
[P0|0104] sensor=humidity value=20.89%
[P0|0105] sensor=humidity value=22.2%
[P0|0106] sensor=humidity value=16.72%
[P0|0107] sensor=humidity value=16.95%
[P2|0188] sensor=pressure value=29.15hPa
[P2|0189] sensor=temperature value=29.11C
[P2|0190] sensor=pressure value=18.85hPa
[P0|0108] sensor=humidity value=31.25%
```

The time difference between the two operations is less than 1 millisecond. By the time you see it print 3 to 4 lines of messages, and your brain tells your finger to press Ctrl + C, the code has already finished executing the `consumer.commit()` calls for those messages. The offset has been successfully saved to Kafka, so when it restarts, it will naturally continue reading the next new message, and we don't see a replay.

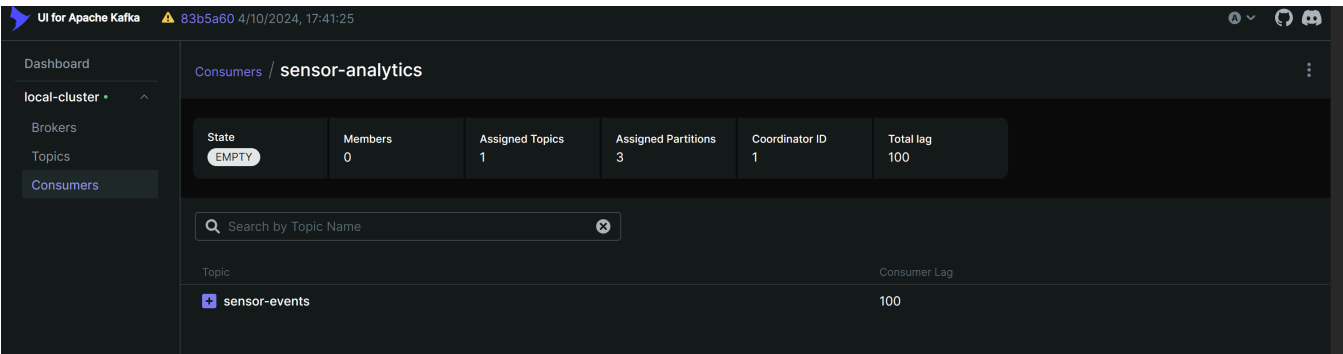
Step 6: Monitor Consumer Lag in Kafka UI

Open <http://localhost:8080> and navigate to Consumer Groups → sensor-analytics.



To understand how to monitor the health of the streaming pipeline visually.

1. Stop all consumers.
2. Run `python producer.py` twice to send 100 messages.
3. Open `http://localhost:8080`, go to Consumer Groups -> `sensor-analytics`.



4. Restart the consumer and watch the UI.

Consumers / sensor-analytics					
State	Members	Assigned Topics	Assigned Partitions	Coordinator ID	Total lag
STABLE	1	1	3	1	0

Topic	Consumer Lag
sensor-events	0

In the UI, the `LAG` column read 100. Once the consumer is restarted in the terminal, the `LAG` value visibly drop to 0.

Reflection Questions

- 1. You used `key=sensor_type` in the producer. What would happen to message ordering if you had used `key=None` instead?**

If `key=None` was used, Kafka would rely on round-robin routing to distribute the messages evenly across all available partitions to maximize throughput. Consequently, readings from the same sensor (e.g., 'temperature') would be scattered across multiple partitions. Since Kafka only guarantees strict ordering within a single partition, the consumer would lose the chronological ordering of the temperature readings.
- 2. With `enable.auto.commit=False`, what are the exact conditions under which a message could be processed twice? What about zero times?**

Processed Twice (Duplicates): If the application successfully processes a message (e.g., updates a database) but crashes before the `consumer.commit()` command executes, the offset remains uncommitted. Upon restart, the consumer will resume from the last known committed offset and replay the message, resulting in it being processed a second time. This is "at-least-once" delivery.

Processed Zero Times (Loss): If the code is structured to call `consumer.commit()` before the business logic processing occurs, and the application crashes immediately after the commit but before the processing is complete, the message is permanently skipped. Upon restart, the consumer will read the next offset, meaning the message was processed zero times. This represents "at-most-once" delivery.
- 3. You have a topic with 6 partitions and a consumer group with 8 consumers. How many consumers are idle? What is the maximum number of consumers that can usefully process this topic in parallel?**

Kafka guarantees that each partition is assigned to at most one consumer within the same consumer group. Therefore, if there are 6 partitions and 8 consumers, 2 consumers will remain idle without any partitions assigned to them. The maximum number of consumers that can usefully process this topic in parallel is strictly equal to the number of partitions, which is 6.
- 4. Explain the difference between `auto.offset.reset=earliest` and `auto.offset.reset=latest`. In which situations would you choose each?**

`auto.offset.reset=earliest`: When the consumer group connects and finds no committed offset, it starts reading from the very first available message in the partition log. You choose this when you want to replay historical data, such as bootstrapping a new database, training a model on past events, or recalculating an aggregate from scratch.

`auto.offset.reset=latest`: When no committed offset exists, the consumer skips all historical messages and only processes new records that arrive after it starts. You choose this for real-time monitoring, live dashboards, or temporary alerting systems where old data is irrelevant to the current operational state.

5. Your producer's throughput is 10000 messages/second but the broker is only acknowledging 2000/second. Which producer parameters would you adjust to improve throughput while keeping `acks=all`?

Increase `linger.ms`: This forces the producer to pause briefly before sending, allowing more messages to accumulate in the memory buffer.

Increase `batch.size`: This raises the maximum byte size of the payload sent per partition (e.g., moving from the default 16384 up to 65536 or 131072).